Performance Assessment of Open Telemetry Tracing in MQ 9.4.4

Objective

To assess the performance impact of enabling an MQ Tracing User Exit that exports telemetry data to a **Jaeger Collector** using OpenTelemetry. The assessment focuses on measuring changes in **message throughput** and **CPU utilization** on the IBM MQ queue manager while tracing is enabled.

The goal is to determine the performance overhead introduced specifically by the IBM MQ OpenTelemetry tracing service—implemented as an IBM MQ API exit (Instana)— and to identify configurations that minimize performance degradation in production environments.

Overview of IBM MQ OpenTelemetry tracing service with Jaeger

- A distributed tracing system that tracks the path and timing of requests across multiple services.
- The MQ Tracing User Exit along with OpenTelemetry client shared libraries, creates spans (trace events) and sends to the Jaeger Collector(in the case of this report) for ingestion and processing.

Performance impact considerations

- **Span creation overhead** inside the MQ Tracing User Exit, which executes within the channel agent (runmqchi).
- **Collector processing and export overhead** for spans sent to Jaeger (local or remote).

Metrics measured

- Message throughput changes with tracing enabled at various Monitor-Levels.
- CPU utilization increases resulting from span creation, encoding, and export performed inside the channel agent.

Refer to the links below for more information about the IBM MQ OpenTelemetry tracing service, IBM MQ tracing exit (Instana) and its usage

https://www.ibm.com/docs/en/ibm-mq/9.4.x?topic=network-opentelemetry-tracing

https://www.ibm.com/docs/en/instana-observability/1.0.307?topic=mq-tracing#enabling-ibm-mq-tracing-user-exit-for-on-premises-ibm-mq

https://www.ibm.com/support/pages/ibm-mq-tracing-support-ibm-mq-tracing-user-exits

The link below gives information about Jaeger installation, configuration etc

https://www.jaegertracing.io/docs/2.dev/deployment/

Test Setup

- Queue Manager: Configured on a dedicated host.
- Applications: Two separate machines host the Requester and Responder applications.
- Queues: 10 queues used by the applications for messaging.
- Connection Mode: Applications connect to the queue manager in client mode.
- Message Size / Type:
  - 2 KB messages: Persistent
  - 2 KB messages: Non-persistent
- Jaeger Collector: Tested both local (same host as MQ) and remote (separate host).
- Monitor-Level Settings: Off, Quiet, Normal, Debug (configured using the IBM MQ Tracing User Exit)
- Hardware / OS:
  - Processor: 2 × 16-core Intel® Xeon® Gold 6544Y CPUs @ 3.60 GHz
  - Memory: 256 GB RAM
  - Operating System: Red Hat Enterprise Linux Server release 9.6 (Plow)

This setup simulates a client-server MQ environment with distributed message flows, providing sufficient resources to assess the performance impact of Jaeger-based tracing under increasing load.

Refer to the link below for more information on workload type: RR-CC

https://github.com/ibm-messaging/mqperf/blob/gh-pages/MQ_V9.4_Performance_Report_xLinux_v1.0.pdf

Methodology / Test Plan

1. Test Variables

- Monitor-Level: Off, Quiet, Normal, Debug (set via IBM MQ Tracing User Exit)
- Collector location: Local vs Remote

2. Test Execution

- Send messages of 2 KB (persistent) and 2 KB (non-persistent) for each Monitor-Level and collector location.
- Metrics captured during the test:

    - Message throughput (messages/sec)
    - CPU utilization (%)
- Test runs continuously for several iterations while gradually increasing the number of Requester applications to observe performance trends under increasing load.

3. Data Analysis & Visualization

- Graph **Throughput vs Monitor-Level** for local and remote collectors.
- Graph **CPU utilization vs Monitor-Level** for local and remote collectors.
- Summarize trends and quantify the overhead introduced by the IBM MQ OpenTelemetry tracing service while processing and sending data to a Jaeger tracing.

Performance Assessment

Persistent

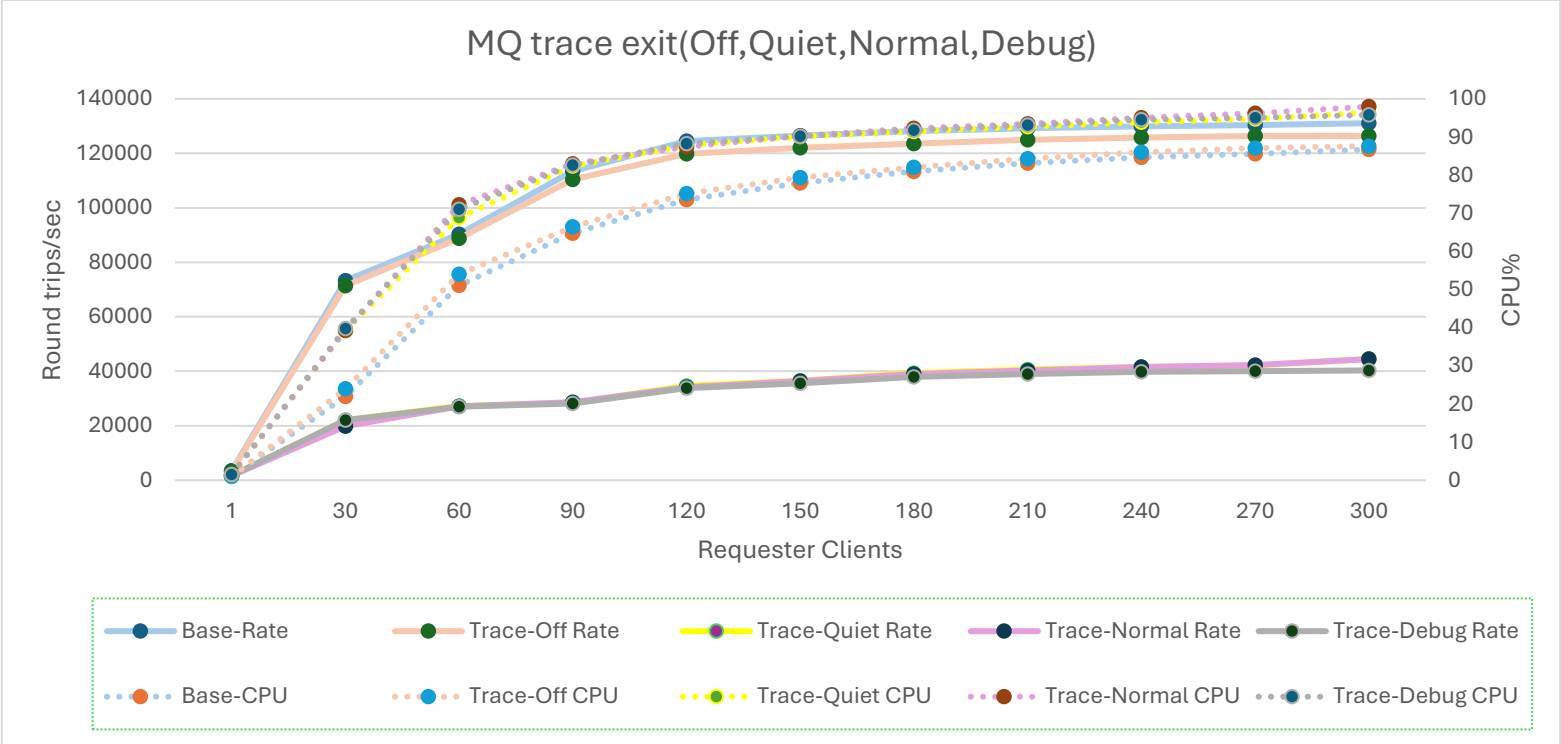

Figure 1 - MQ Exit with Monitor-Level = Off, Quiet, Normal, Debug while running application (RR-CC) for Persistent messaging(2K) and Jaeger as a local collector

Legend	***Description***	***Corresponding CPU Usage***
Base Rate	*No Activity Trace*	*Base CPU*
Trace-Off Rate	*Trace Exit enabled for Off*	*Trace-Off CPU*
Trace-Quiet Rate	*Trace Exit enabled for Quiet*	*Trace-Quiet CPU*
Trace-Normal Rate	*Trace Exit enabled for Normal*	*Trace-Normal CPU*
Trace-Debug Rate	*Trace Exit enabled for Debug*	*Trace-Debug CPU*

Trace Scenarios	Round trips/sec	CPU%	Throughput Impact%
Without *Trace Exit*	131074	87	
Trace Exit =Off	126371	87	–4
Trace Exit =Quiet	44516	96	-66
Trace Exit =Normal	44465	97	-66
Trace Exit =Debug	40296	96	-69

- The table gives the throughput for various Trace Scenarios when Requester Clients reaches to 300

- **Trace Exit = Off** introduces *negligible overhead*, with only a **4% throughput reduction** and no increase in CPU usage.
- **Trace Exit = (Quiet, Normal, Debug)** causes a **significant throughput drop of 66–69%**, showing substantial processing overhead once span generation is enabled.
- Overhead of **Quiet and Normal levels** is very similar.
- **Debug level** produces the *highest tracing volume* and results in the **lowest throughput**.

Non-Persistent

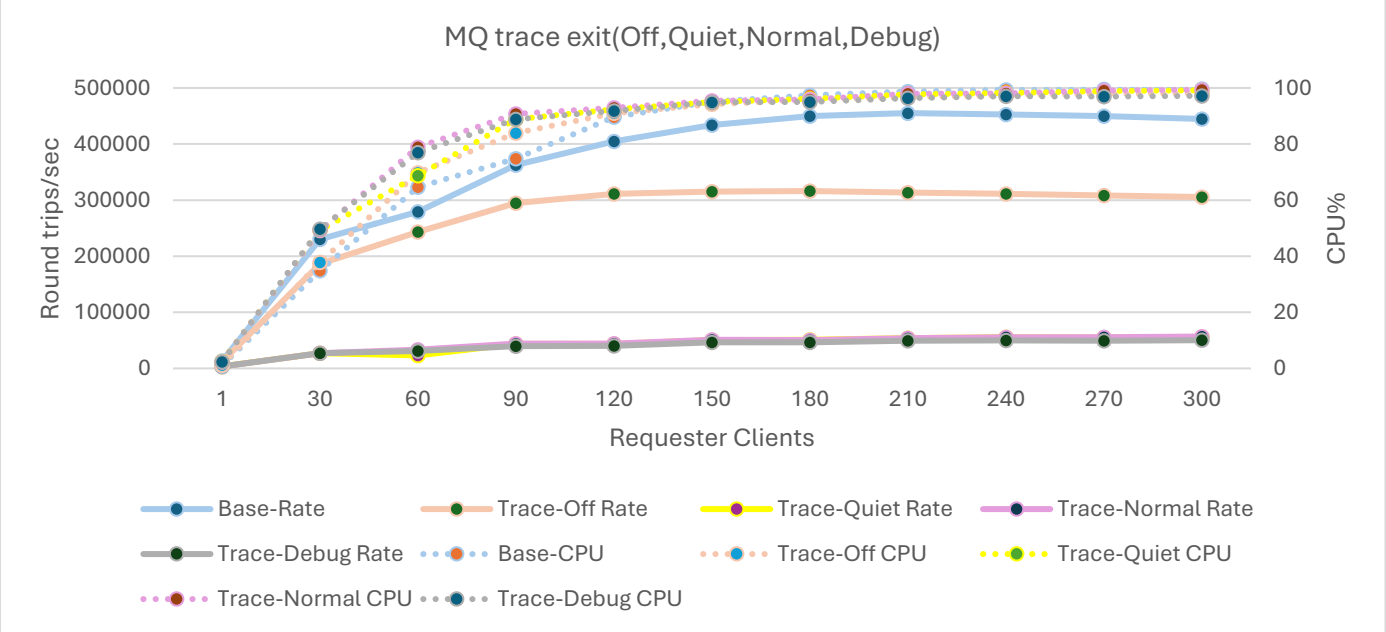

Figure 2 - MQ Exit with Monitor-Level = Off, Quiet, Normal, Debug while running application (RR-CC) for Non-Persistent messaging(2K) and Jaeger as a local collector

Legend	***Description***	***Corresponding CPU Usage***
Base Rate	*No Activity Trace*	*Base CPU*
Trace-Off Rate	*Trace Exit enabled for Off*	*Trace-Off CPU*
Trace-Quiet Rate	*Trace Exit enabled for Quiet*	*Trace-Quiet CPU*
Trace-Normal Rate	*Trace Exit enabled for Normal*	*Trace-Normal CPU*
Trace-Debug Rate	*Trace Exit enabled for Debug*	*Trace-Debug CPU*

Trace Scenarios	**Round trips/sec**	**CPU%**	**Throughput Impact%**
Without *Trace Exit*	444635	99	
Trace Exit =Off	305272	99	–31
Trace Exit =Quiet	56259	99	-87
Trace Exit =Normal	56740	99	-87
Trace Exit =Debug	50246	97	-88

- The table gives the throughput for various Trace Scenarios when Requester Clients reaches to 300

- **Trace Exit = Off** results in a **moderate throughput reduction (~31%)** with CPU near 99%, indicating minimal overhead when the exit is present but not actively tracing.
- **Trace Exit = Quiet, Normal, and Debug** cause **severe throughput drops (~87–88%)**, reflecting significant processing overhead when span generation is enabled.
- All active tracing modes (Quiet, Normal, Debug) push the CPU close to saturation, showing that the system becomes CPU-bound as soon as tracing starts, regardless of verbosity level.
- Increasing trace detail from Quiet → Normal → Debug has **little additional impact on throughput**, suggesting the system is already fully utilized once tracing is enabled.
- Debug mode has slightly lower throughput than Quiet, but the difference is minor, indicating that additional debug information adds only marginal extra cost once CPU saturation is reached.

Comparison of Jaeger as a remote and local collector

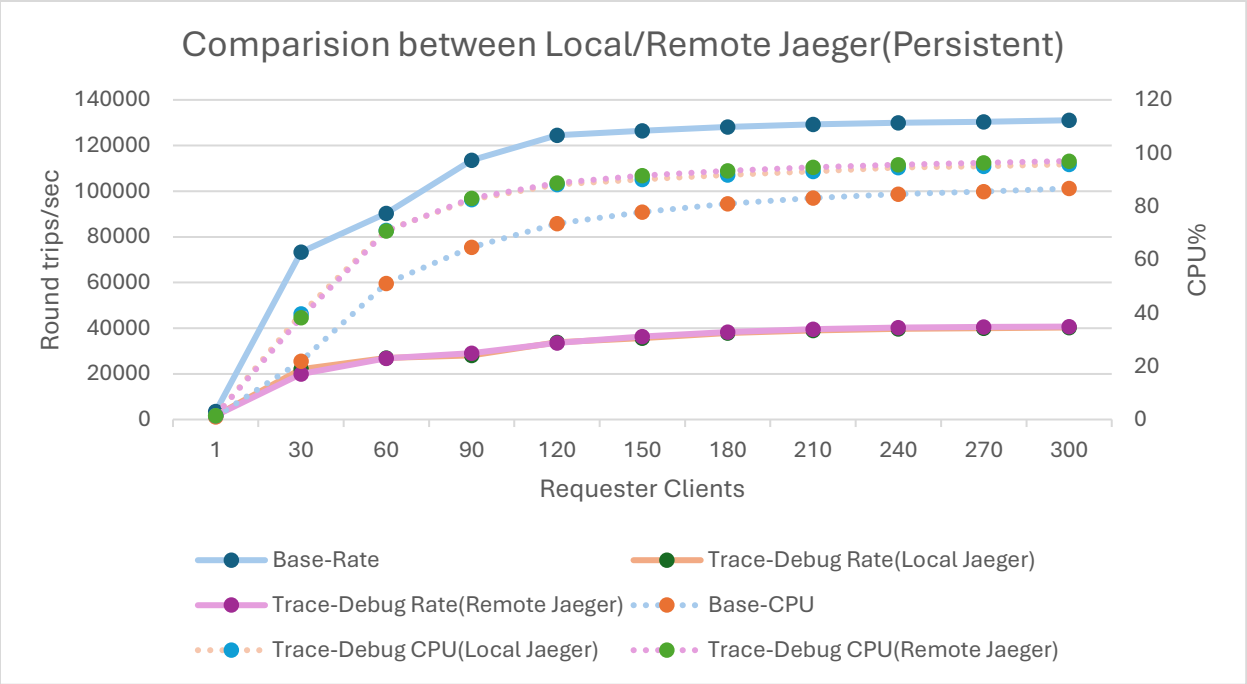

Figure 3 - Jaeger-MQ trace Exit with Monitor-Level = Debug while running application (RR-CC) for Persistent messaging(2K) – Jaeger as a Local and Remote collector

Legend	***Description***	***Corresponding CPU Usage***
Base Rate	*No Activity Trace*	*Base CPU*
Trace-Debug Rate(Local Jaeger)	*Trace Exit enabled for Debug and Jaeger as a Local Collector*	*Trace-Debug CPU(Local Jaeger)*
Trace-Debug Rate(Remote Jaeger)	*Trace Exit enabled for Debug and Jaeger as a Remote Collector*	*Trace-Debug CPU(Remote Jaeger)*

Trace Scenarios	**Round trips/sec**	**CPU%**	**Throughput Impact%**
Without *Trace Exit*	131074	87	
Trace-Debug Rate(Local Jaeger)	40296	96	–69
Trace-Debug Rate(Remote Jaeger)	40709	97	-69

The comparison shows **negligible difference** between Local and Remote Jaeger collectors in Debug mode.
This indicates:

- MQ performance degradation is dominated by **exit processing and span generation**, not network latency.
- Exporting spans to a remote Jaeger collector does **not** add noticeable overhead.

- The system remains **CPU-saturated** in both cases, masking any small network-related delays.

Conclusion

- **Active Trace Levels (Quiet, Normal, Debug)**

  Persistent scenario:

  - Throughput drops by **66–69%**, indicating substantial overhead once tracing is enabled.
  - Quiet and Normal modes show nearly identical throughput, confirming the system reaches saturation immediately after tracing starts.
  - Debug mode generates the highest trace volume and results in the **lowest throughput**, showing additional overhead compared to Quiet/Normal.

  Non-Persistent scenario:

  - Throughput degradation is **more severe (~87–88%)**, demonstrating significantly higher sensitivity to tracing when persistent is disabled.
  - Quiet, Normal, and Debug modes all drive the system to **CPU saturation**, regardless of trace verbosity.
  - Debug mode performs slightly worse than Quiet/Normal, but the difference remains marginal once CPU saturation is reached.

- **CPU Profiling Observations**
  - With Monitor-Level = Quiet, Normal, Debug, **runmqchi execution path** accounts for ~55% of CPU samples
  - Stack clearly shows **MQ Trace Exit functions** and **OpenTelemetry client shared libraries** executing within channel processing flow
  - At Monitor-Level = Off, these stacks are **absent**, confirming CPU overhead comes from **active span generation and export**

- **Local vs Remote Jaeger (Debug mode)**
  - Negligible difference in throughput or CPU usage
  - Performance degradation dominated by **exit processing and span generation**, not network latency
  - Export to remote collector does **not add noticeable overhead**
  - CPU saturation masks any small network-related delays